

Author
Student's Name

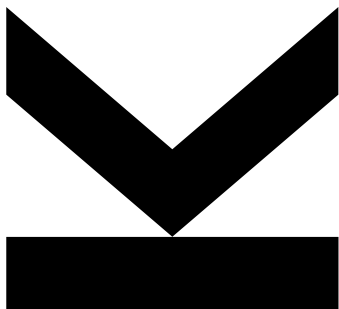
Submission
Departement Name

Thesis Supervisor
Professor's Name

Assistant Thesis Supervisor
Assistant's Name

Month Year
(For Information: Month and
year of submission to
Examination and Recognition
Services)

Thesis Title



Bachelor's Thesis
to confer the academic degree of
Bachelor of Science
in the Bachelor's Program
Informatik

Bachelor's Thesis

**Coco/R Extension for Visual Studio Code:
Syntax highlighting and workflow-supporting features**

Dipl.-Ing. Markus Weninger
Institute for System Software

Student: Byambabayar Altangerel

Advisor: Dipl.-Ing. Markus Weninger, BSc.

Start date: February 2021

P +43-732-2468-4361
F +43-732-2468-4345
markus.weninger@jku.at

Coco/R is a compiler generator, which takes an attributed grammar (written in Cocol/R) of a source language and generates a scanner and a parser for this language. Many projects use Coco/R to parse data, for example logs, configurations, and user input. It is also used to develop domain-specific languages.

While Cocol/R is relatively easy to learn and has a well-written user manual¹, it encompasses certain syntactical patterns that would profit from IDE support such as syntax highlighting or region collapsing. Such IDE support has been developed in the past for IDEs such as Eclipse or Netbeans.

The goal of this thesis is to develop an extension² for Visual Studio Code (VSC) to provide a better experience when working with Coco/R. VSC provides the possibility to add support for new programming languages³, which can be splitted into *declarative language features* such as syntax highlighting, bracket autoclosing, auto indentation or snippet completion and *programmatic language features* such as auto completion or jump to definition operations. This thesis focuses on the first of the two.

In addition to implementing these declarative language features (which are performed using tokenization based on a TextMate grammar, which the student has to translate from the official Cocol/R EBNF), various workflow-supporting features should be implemented:

- A context menu to "Run" the currently opened .atg file (i.e., to generate a scanner and parser for the given attributed grammar)⁴.
- The user should be able to specify workspace-specific settings - especially a output directory (i.e., where the scanner and parser files should be generated) and the package definition (i.e., in which package the generated scanner and parser should reside).
- Scanner.frame and Parser.frame files (needed by Coco/R during the generation process) should be automatically created if they do not exist yet.

Even though Coco/R can be used to generate scanners and parsers in various languages, the extension developed in this thesis only has to support Java. Java code should be formatted and highlighted according to Visual Studio Code's default Java settings (which is supported by VSC via *embedded languages*). Alignment of annotated Java code would be nice-to-have, but are not mandatory.

Modalities:

The progress of the project should be discussed at least every two weeks with the advisor. A time schedule and a milestone plan must be set up within the first 3 weeks. It should be continuously refined and monitored to make sure that the thesis will be completed in time. The final thesis must be submitted not later than 16.08.2021.

¹ <https://ssw.jku.at/Research/Projects/Coco/Doc/UserManual.pdf>

² <https://code.visualstudio.com/api>

³ <https://code.visualstudio.com/api/language-extensions/overview>

⁴ Example Coco/R call: `java -jar Coco.jar MyFile.atg -o /my/output/folder -package at.jku.coco`

Abstract

The Algorithms and Data Structures 2 course at the Institute for Pervasive Computing has traditionally relied on static slides to teach concepts such as graph theory and algorithm visualization. While effective for conveying information, this format limits opportunities for students to engage interactively with the material and for teachers to demonstrate concepts dynamically.

Graph theory and algorithms are foundational topics in computer science, and their inherently dynamic nature makes them ideal for interactive exploration. To address this, this thesis presents a web-based application that enables both students and teachers to create graphs and simulate algorithms in a hands-on, intuitive way.

This document details the design, development, and testing of the application, focusing on its usability and features tailored to an educational setting. The result is a practical tool that enhances learning by allowing students to experiment with concepts and empowers teachers to demonstrate algorithms in a more engaging and flexible manner.

Kurzfassung

Der Kurs Algorithmen und Datenstrukturen 2 am Institut für Pervasive Computing hat traditionell statische Folien verwendet, um Konzepte wie Graphentheorie und Algorithmusvisualisierung zu vermitteln. Obwohl diese Methode zur Wissensvermittlung effektiv ist, bietet sie nur begrenzte Möglichkeiten für Studierende, interaktiv mit dem Material zu arbeiten, und für Lehrende, die Konzepte dynamisch zu demonstrieren.

Graphentheorie und Algorithmen sind grundlegende Themen der Informatik, und ihre dynamische Natur macht sie ideal für interaktive Erkundungen. Um dies zu ermöglichen, wird in dieser Arbeit eine webbasierte Anwendung vorgestellt, die sowohl Studierenden als auch Lehrenden erlaubt, eigene Graphen zu erstellen und Algorithmen intuitiv und praxisnah zu simulieren.

Diese Arbeit beschreibt den Entwurf, die Entwicklung und die Testphasen der Anwendung mit einem besonderen Fokus auf Benutzerfreundlichkeit und Funktionen, die speziell auf den Einsatz im Bildungsbereich zugeschnitten sind. Das Ergebnis ist ein praktisches Werkzeug, das das Lernen fördert, indem es Studierenden ermöglicht, Konzepte experimentell zu erforschen, und Lehrende dabei unterstützt, Algorithmen ansprechender und flexibler zu demonstrieren.

Table of Content

Contents

1 Introduction

Graphs are a fundamental concept in computer science, with applications spanning a wide range of fields, including network analysis, optimization, and algorithm design. Effective teaching and learning of graph theory and graph-based algorithms often require the use of tools that allow users to interactively create, visualize, and explore graphs. However, many traditional approaches rely on static representations, such as slides or diagrams, which limit the potential for hands-on engagement and dynamic exploration of these concepts.

A key challenge lies in providing students and educators with tools that are both accessible and versatile. While some graph visualization tools exist, they often lack the flexibility required for a comprehensive educational experience, particularly when it comes to seamlessly integrating graph creation, exploration, and algorithm visualization. This limitation creates an opportunity for a unified, user-friendly application tailored specifically for educational purposes.

This thesis contributes to addressing this challenge by presenting the development of a web-based application that supports interactive graph creation and algorithm visualization. The contributions of this work include:

1. A **graph builder** for creating and modifying both directed and undirected graphs.
2. A **visualizer** for displaying and exploring graphs and their properties.
3. Algorithm visualizations for **Dijkstra's shortest path**, **breadth-first search (BFS)**, and **depth-first search (DFS)** on existing graphs.
4. Predefined example graphs, which can be opened in the builder, visualizer, or directly in the algorithm simulation mode for hands-on experimentation.

During the development process, initial work began with Graphviz, but limitations in flexibility and interactivity led to a switch to **D3.js**, a more powerful library for creating interactive visualizations. The project progressed iteratively, starting with a prototype graph builder in D3.js to familiarize with the library, followed by stepwise development of the graph visualizer and algorithm simulations. Finally, predefined examples were incorporated to demonstrate the tool's capabilities and provide immediate utility for educational purposes.

In summary, this chapter outlines the task and approach taken to address it. The task at hand involved creating an interactive educational tool that belongs to the fields of graph theory and algorithm visualization. By the conclusion of this work, the requirements of the task were not only fulfilled but also extended through the development of a flexible and versatile application that enhances both teaching and learning experiences in this domain.

2 Background

In this section, the theoretical and technical foundations relevant to this work are presented, providing the reader with the necessary background knowledge to better understand the development and functionality of the web-based graph application. This includes an overview of graph theory, algorithm visualization, and the technologies used for implementation.

2.1 Graph Theory and Algorithms

Graphs are abstract data structures used to model pairwise relationships between objects, with applications in numerous domains such as network analysis, route optimization, and scheduling. A graph consists of vertices (or nodes) connected by edges, which can be either directed or undirected. Key algorithms commonly applied to graphs include:

- **Breadth-First Search (BFS):** An algorithm for traversing or searching graph data structures level by level.
- **Depth-First Search (DFS):** An algorithm for traversing graphs by exploring as far as possible along a branch before backtracking.
- **Dijkstra’s Algorithm:** An algorithm for finding the shortest paths between nodes in a weighted graph.

These algorithms form the core functionality of the application presented in this work and are visualized interactively to enhance user understanding.

2.2 Algorithm Visualization in Education

Algorithm visualization has long been a valuable tool in computer science education. By visually representing the step-by-step execution of an algorithm, it helps learners to understand its inner workings and behavior on various graph structures. However, many existing tools are limited in their interactivity or require significant setup, making them less accessible for immediate classroom use.

2.3 Technology Stack

The application developed in this work leverages modern web technologies to provide a flexible and interactive platform:

- **D3.js:** A JavaScript library for creating data-driven visualizations, chosen for its flexibility and ability to render interactive graph representations.
- **TypeScript:** A strongly typed superset of JavaScript that enhances development efficiency and reliability by enabling static type checking.
- **Svelte:** A modern front-end framework used to build highly reactive and performant user interfaces. Its compile-time approach reduces runtime overhead.
- **Vite:** A fast build tool and development server designed to optimize front-end development workflows.

- **npm:** A package manager used for handling project dependencies and ensuring seamless integration of third-party libraries.

Initially, Graphviz was considered for implementing the visualization. While it excels in static graph rendering, it lacks the interactive capabilities required for this project. Consequently, D3.js was adopted as it provides greater flexibility and control over interactive elements. Svelte and Vite further streamline the development process, ensuring a responsive and user-friendly interface.

2.4 Applications and Educational Context

The primary focus of this application is in education, particularly in teaching graph theory and algorithms. By providing an environment where users can build, visualize, and experiment with graphs, the tool serves as both a learning aid for students and a teaching aid for educators. This interactive approach fosters engagement and supports deeper comprehension of graph-based concepts.

In the following sections, this foundation is built upon to describe the design and implementation of the application, highlighting how these theoretical concepts and technologies were applied to create a practical and effective educational tool.

3 Overview

The purpose of this web-based application is to provide an interactive platform for creating, visualizing, and simulating graphs and algorithms, with a focus on enhancing the educational experience for both students and educators. This chapter outlines the system's main features, its overall architecture, and the interaction between its components.

3.1 Purpose and Key Features

The application is designed to bridge the gap between theoretical learning and practical experimentation in graph theory and algorithm visualization. Its key features include:

- A **graph builder** for creating and modifying directed and undirected graphs through an intuitive interface.
- A **graph visualizer** that displays graphs and their properties dynamically.
- Algorithm visualizations for **Dijkstra's shortest path**, **breadth-first search (BFS)**, and **depth-first search (DFS)**.
- Predefined example graphs that can be loaded for immediate use in the builder, visualizer, or algorithm simulation modes.
- A lightweight and responsive interface, accessible through modern web browsers without additional installation.

The system is primarily intended for use in educational contexts, where users can create their own graphs or use predefined examples to explore graph algorithms interactively. While the tool focuses on graph visualization and algorithm simulation, it does not cover advanced topics such as graph embeddings or probabilistic graph models.

3.2 System Architecture

The application is structured into several interconnected subsystems, each responsible for a specific aspect of functionality:

1. **Frontend:** Built with Svelte, the frontend provides an intuitive and interactive user interface for creating, modifying, and visualizing graphs. It communicates with the visualization engine and ensures real-time updates.
2. **Visualization Engine:** Powered by D3.js, this subsystem handles the rendering of graphs and animations for algorithm simulations. It supports smooth interactions, such as dragging nodes or adjusting edges.
3. **Algorithm Module:** Implemented in TypeScript, this module performs the computations for algorithms such as Dijkstra, BFS, and DFS. It interacts with the visualization engine to display step-by-step execution.
4. **Example Loader:** A component that allows users to load predefined graph examples. It integrates seamlessly with the builder and algorithm visualizer.

3.3 Interaction and External Dependencies

The system is designed to run entirely within a web browser, eliminating the need for external backend servers. It depends on modern web technologies, including:

- **D3.js:** For dynamic graph rendering and interactive visualizations.
- **TypeScript:** For robust and reliable implementation of algorithms and application logic.
- **Svelte and Vite:** For building a reactive and highly performant user interface.
- **npm:** For managing dependencies and ensuring compatibility with third-party libraries.

3.4 System Interaction and Workflow

The application workflow is designed to be user-centric:

- Users begin by either creating a new graph from scratch using the graph builder or loading a predefined example.
- Once the graph is created or loaded, users can visualize its structure and properties.
- For algorithm simulations, users can select an algorithm (e.g., Dijkstra, BFS, or DFS) and observe its execution step-by-step on the graph.
- The interactive interface allows users to adjust the graph dynamically and rerun simulations as needed.

3.5 Limitations and Scope

While the application provides robust functionality for graph creation, visualization, and algorithm simulation, it is not designed for advanced computational tasks such as solving large-scale optimization problems or analyzing massive datasets. Its scope is intentionally focused on education, making it simple and accessible for students and educators.

In summary, the system provides a versatile platform for exploring graph theory and algorithms interactively. The architecture is designed to ensure modularity, responsiveness, and ease of use, enabling seamless integration of its various components to deliver an intuitive user experience.

4 Implementation

4.1 Early Exploration

The development process began with an exploration of available tools for graph visualization. Initial prototyping was conducted using Graphviz, a popular tool for static graph rendering. While Graphviz proved effective for creating static visualizations, it posed significant limitations for this project:

- Lack of interactivity and flexibility required for dynamic visualization and user interaction.
- Limited compatibility with modern web technologies for building highly interactive applications.

As a result, the decision was made to switch to **D3.js**, a JavaScript library that offers robust support for dynamic and interactive graph rendering. Despite its steep learning curve, D3.js provided the flexibility and control needed to meet the project's requirements. Early experimentation focused on understanding its core concepts, including data binding, SVG manipulation, and event handling.

4.2 Stepwise Implementation

4.2.1 Prototype Graph Builder

The first implementation step involved creating a basic prototype for the graph builder using D3.js. This included:

- Dynamic addition and deletion of nodes and edges.
- Learning to manipulate nodes and edges interactively through drag-and-drop functionality.
- Initial usability testing to refine the interface and ensure user-friendly interactions.

4.2.2 Directed and Undirected Graph Creation

Following the prototype, support for both directed and undirected graphs was implemented. Key considerations included:

- Visual differentiation between directed and undirected edges.
- Functional adjustments to ensure the correct handling of graph types in algorithms.
- Enhancements to the user interface for toggling between directed and undirected graphs.

4.2.3 Graph Visualizer

The next step was the development of a dedicated visualization component to display and interact with graphs. Features implemented included:

- Interactive manipulation of nodes and edges.
- Real-time updates to the graph as modifications were made.
- Initial integration of graph traversal highlighting to support future algorithm simulations.

4.2.4 Algorithm Simulations (Dijkstra, BFS, DFS)

With the visualizer in place, algorithm simulation functionality was added:

- Step-by-step execution of Dijkstra’s algorithm to visualize the shortest path.
- BFS and DFS traversal simulations to demonstrate graph exploration.
- Integration with the visualizer to highlight nodes and edges during algorithm execution.

Special attention was given to making the simulations user-friendly by providing controls for pausing, resuming, and resetting the algorithms.

4.2.5 Integration of Predefined Examples

To showcase the application’s capabilities, predefined example graphs were integrated. These examples allow users to:

- Load ready-made graphs directly into the builder or visualizer.
- Experiment with algorithm simulations without needing to create graphs from scratch.
- Explore use cases, such as road networks or tree structures, to understand graph algorithms in context.

4.3 Challenges and Iterations

The development process involved overcoming several challenges:

- **Learning Curve:** Adapting to D3.js required significant effort, particularly for implementing advanced interactions and animations.
- **Performance Issues:** Handling larger graphs required optimization of rendering and interaction logic to maintain smooth performance.
- **User Interface Design:** Ensuring the application was intuitive and user-friendly involved multiple iterations and feedback from early testers.

Each challenge was addressed through iterative development, with frequent testing and refinement to ensure a high-quality final product.

5 Evaluation / Usage

The final system is a web-based application designed to provide an intuitive, interactive, and accessible platform for creating, visualizing, and simulating graphs and algorithms. This section outlines the system’s features, architecture, and usability, highlighting how the components integrate to deliver the intended functionality.

5.1 Features

The system includes the following key features:

- **Graph Builder:** Users can create, edit, and manipulate directed and undirected graphs. Nodes and edges can be added or deleted dynamically, with support for interactive drag-and-drop adjustments.
- **Graph Visualizer:** A dedicated visualizer displays the graph, allowing users to explore its structure and properties interactively.
- **Algorithm Simulations:** Step-by-step visualizations for:
 - **Dijkstra’s Algorithm:** Shortest path calculation.
 - **Breadth-First Search (BFS):** Level-wise graph traversal.
 - **Depth-First Search (DFS):** Recursive graph traversal.
- **Predefined Examples:** Users can load example graphs for exploration, visualization, and algorithm simulation without needing to create graphs from scratch.
- **Responsive User Interface:** The application is accessible through modern web browsers, offering a seamless experience across devices.

5.2 Integration and Workflow

The system workflow is designed to be user-friendly and intuitive:

- Users start by either creating a new graph or loading a predefined example.
- The graph builder allows dynamic creation and modification of nodes and edges.
- The graph visualizer provides real-time feedback and displays the graph structure interactively.
- Users can run algorithm simulations on the graph, observing step-by-step execution and real-time animations.
- Users can reset or modify the graph and rerun simulations as needed.

5.3 Usability and Educational Impact

The system is tailored for educational use, benefiting both students and educators:

- **For Students:** The interactive graph builder and algorithm simulations provide a hands-on learning experience, helping to reinforce theoretical concepts through experimentation.
- **For Educators:** The tool serves as a teaching aid, enabling dynamic demonstrations of graph algorithms and their behavior in real-time.

The system's responsive design ensures accessibility across different devices, making it suitable for classroom use, remote learning, and self-study.

5.4 Limitations and Future Enhancements REDUNDANT, MAKE PART OF FUTURE WORK

While the system achieves its primary goals, there are some limitations:

- **Scalability:** Performance may degrade with very large graphs due to browser limitations.
- **Algorithm Coverage:** Currently, only a subset of graph algorithms is implemented (Dijkstra, BFS, DFS). Expanding the algorithm library would enhance the system's utility.
- **Backend Integration:** The application is entirely frontend-based, and adding backend support could enable features such as saving/loading graphs or collaborative editing.

These limitations offer opportunities for future improvements and extensions to the system.

5.5 Summary REDUNDANT, OWN CHAPTER

The final system provides an effective and versatile platform for exploring graph theory and algorithms interactively. Its modular architecture, dynamic features, and educational focus make it a valuable tool for students and educators alike. By combining flexibility, usability, and interactivity, the application bridges the gap between theoretical knowledge and practical learning in graph-based concepts.

6 Related Work

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

7 Conclusions and Future Work

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

List of Tables

List of Figures

Listings

	<i>Curriculum Vitae</i> Dipl.-Ing. Dr. Markus Weninger, BSc 📍 4113 St. Martin im Mühlkreis, Schulstraße 24 ✉ markus.weninger@jku.at ☎ +43660/3115418
Birth Date	04.04.1992 in Vöcklabruck, Austria
Education	2017 - 2021: PhD - Computer Science (with distinction) Johannes Kepler University Linz Thesis: Detection and Analysis of Memory Anomalies in Managed Languages Using Trace-Based Memory Monitoring 2015 - 2017: Master - Computer Science / Software Engineering (with distinction) Johannes Kepler University Linz Thesis: User-defined Classification and Multi-grouping of Data in a Memory Monitoring Tool 2012 - 2015: Bachelor - Informatik (with distinction) Johannes Kepler University Linz Thesis: An Experiment to Measure the Performance Trade-off between Traditional IO and Memory-mapped Files 2006 - 2011: HTL - Higher education (with distinction) HTL Leonding, EDV & Organisation (Upper Secondary Technical and Vocational College - Department for Informatics) Thesis: Digital Teaching System – Digitales Lehrsystem an der HTL Leonding
Professional Experience	Since 10/2021: Senior Lecturer (for Computer Science) Johannes Kepler University, Institute for System Software – Linz 09/2017 – 08/2021: Institute Assistant / Researcher Johannes Kepler University, Institute for System Software – Linz 09/2017 – 01/2020: Researcher Johannes Kepler University, Christian Doppler Laboratory MEVSS - Linz 10/2015 – 08/2017: Student Researcher Johannes Kepler University, Christian Doppler Laboratory MEVSS – Linz 05/2013 – 09/2015: Software Engineer C#/.Net bet-at-home.com Entertainment GmbH – Linz 07/2010 – 08/2010: Intern - Software Engineer MIC Customs Solutions / MIC Datenverarbeitung GmbH – Linz 07/2008 - 08/2008: Intern – IT AIM Technical Solutions GmbH – Timelkam
Teaching Experience	Algorithmen und Datenstrukturen 2 Algorithms and Datastructures 2 2021 WS – German / English Grundlagen der Programmierung Basics of Software Development 2017 WS - German Programming in Kotlin 2021 WS – English Seminar in Software Engineering: Memory Monitoring and Analysis (2022 SS) – German / English Software Engineering für Jurist*innen Software Engineering for Lawyers (2022 SS) – German Softwareentwicklung 1 Software Development 1 2020 WS, 2021 WS, (2022 SS) – German Softwareentwicklung 2 Software Development 2 2018 SS, 2019 SS, 2020 SS, 2021 SS, (2022 SS) – German Übersetzerbau Compiler Construction 2018 WS, 2019 WS, 2020 WS, 2021 WS – German / English Thesis supervision (Bachelor thesis, Master thesis, Master project): 13 bachelor theses, 6 master theses, 6 master software projects (7.5 ECTS) <i>WS = Winter Semester, SS = Summer Semester all at Johannes Kepler University Linz</i>

Reserach community	Student Volunteers Chair: ManLang 2018 Reviewer: VISSOFT 2021, Journal for “Information and Software Technology” 2022
Languages	German: native English: fluent
Workshops	Events: CoderDojo, Frauen in die Technik, JKU Science Holidays, KinderUni, Tomorrow’s Experts in Computing, Traumberuf Technik, Young Computer Scientists, ... and others ... Topics: JavaFX Game Programming 3D Game Programming Kreatives Programmieren mit micro:bit Creative Programming with micro:bit Spieleentwicklung mit Scratch Game Programming with Scratch Spielerisches Kennenlernen von Sortieralgorithmen Sorting is Fun ... and others ...
Volunteer Work	Since 2021: Rotaract Europe (ERIC – European Rotaract Information Centre) Social and community service, Europe-wide ERIC IT Committee 2021/22 Since 2018: Rotaract KidsCamp Österreich Summer camp for kids from underprivileged families, St. Georgen / Attergau Organizer “Kids and Sponsorships” 2019-2022 Since 2018: Rotaract Social and community service, Linz President 2019/20 President 2020/21 Secretary 2021/22 Social Project Officer 2022/23 Since 2018: Students Union "PhD Studies - Engineering & Natural Sciences" Johannes Kepler University Linz 2015 - 2021: Students Union "Informatik & AI" Johannes Kepler University Linz
Awards	2021: Award of Excellence Dissertation award by the Austrian Federal Ministry of Education, Science and Research 2020: Best Paper "Memory Leak Analysis using Time-Travel-based and Timeline-based Tree Evolution Visualizations", Conference on Smart Tools and Applications in Graphics (STAG) 2020: Best Paper "Memory Cities: Visualizing Heap Memory Evolution Using The Software City Metaphor", IEEE Working Conference on Software Visualization (VISSOFT) 2018: Best Presentation of Best Paper Candidates "User-defined Classification and Multi-level Grouping of Objects in Memory Monitoring", International Conference on Performance Engineering (ICPE) 2018: Best Paper Candidate "User-defined Classification and Multi-level Grouping of Objects in Memory Monitoring", International Conference on Performance Engineering (ICPE) 2017: 2nd place of the Adolf-Adam Price (price for the best informatics master thesis) Johannes Kepler University Linz 2015: Winner of the Coding 4 a Cause (C4C:EU) Contest Association for the Advancement of Assistive Technology in Europe (AAATE) Conference Since 1015: 10.000+ reputation on StackOverflow Online, https://stackoverflow.com/users/2938364/markus-weninger Since 2013: Various "Top 10%" results at Catalysts / Cloudflight Coding Contests Linz / Vienna / Online



Publications

Dipl.-Ing. Dr. Markus Weninger, BSc.

📍 4113 St. Martin im Mühlkreis, Schulstraße 24 ✉ markus.weninger@jku.at 📞 +43660/3115418

Publications

- [18] **Weninger, M.**; Gander, E.; Mössenböck, H.,
"Guided Exploration: A Method for Guiding Novice Users in Interactive Memory Monitoring Tools", ACM HCI (EICS) 2021
- [17] **Weninger, M.**; Makor, L.; Mössenböck, H.,
"Memory Leak Analysis using Time-Travel-based and Timeline-based Tree Evolution Visualizations", STAG 2020 (Best Paper)
- [16] **Weninger, M.**; Makor, L.; Mössenböck, H.,
"Heap Evolution Analysis Using Tree Visualizations", SSP 2020
- [15] **Weninger, M.**; Gander, E.; Mössenböck, H.,
"Investigating High Memory Churn via Object Lifetime Analysis to Improve Software Performance", SSP 2020
- [14] **Weninger, M.**; Makor, L.; Mössenböck, H.,
"Memory Cities: Visualizing Heap Memory Evolution Using the Software City Metaphor", VISSOFT 2020 (Best Paper)
- [13] **Weninger, M.**; Grünbacher, P.; Gander, E.; Schörgenhumer, A.,
"Evaluating an Interactive Memory Analysis Tool: Findings from a Cognitive Walkthrough and a User Study", ACM HCI (EICS) 2020
- [12] **Weninger, M.**; Makor, L.; Mössenböck, H.,
"Memory Leak Visualization using Evolving Software Cities", SSP 2019
- [11] **Weninger, M.**; Gander, E.; Mössenböck, H.,
"Detection of Suspicious Time Windows in Memory Monitoring", MPLR 2019
- [10] **Weninger, M.**; Makor, L.; Mössenböck, H.,
"AntTracks TrendViz: Configurable Heap Memory Visualization Over Time", ICPE 2019
- [9] **Weninger, M.**; Gander, E.; Mössenböck, H.,
"Analyzing Data Structure Growth Over Time to Facilitate Memory Leak Detection", ICPE 2019
- [8] **Weninger, M.**; Grünbacher, P.; Zhang, H.; Yue, T.; Ali, S.,
"Tool Support for Restricted Use Case Specification: Findings from a Controlled Experiment", APSEC 2018
- [7] **Weninger, M.**; Gander, E.; Mössenböck, H.,
"Analyzing the Evolution of Data Structures in Trace-Based Memory Monitoring", SSP 2018
- [6] **Weninger, M.**; Gander, E.; Mössenböck, H.,
"Utilizing Object Reference Graphs and Garbage Collection Roots to Detect Memory Leaks in Offline Memory Monitoring", ManLang 2018
- [5] **Weninger, M.**; Mössenböck, H.,

	"User-defined Classification and Multi-level Grouping of Objects in Memory Monitoring", ICPE 2018 (Best Paper Candidate) [4] Weninger, M.; Lenggauer, P.; Mössenböck, H., "User-centered Offline Analysis of Memory Monitoring Data", ICPE 2017 [3] Lenggauer, P.; Bitto, V.; Mössenböck, H.; Weninger, M., "A Comprehensive Java Benchmark Study on Memory and Garbage Collection Behavior of DaCapo, DaCapo Scala, and SPECjvm2008", ICPE 2017 [2] Lenggauer, P.; Bitto, V.; Fitzek, S.; Weninger, M.; Mössenböck, H., "Efficient Memory Traces with Full Pointer Information", PPPJ 2016 [1] Weninger, M.; Ortner, G.; Hahn, T.; Drümmer, O.; Miesenberger, K., „ASVG - Accessible Scalable Vector Graphics: intention trees to make charts more accessible and usable“, Journal of Assistive Technologies, Vol. 9 Issue 4, 2015
--	--

Statutory Declaration

I hereby declare that the thesis submitted is my own unaided work, that I have not used other than the sources indicated, and that all direct and indirect sources are acknowledged as references.

This printed thesis is identical with the electronic version submitted.

Place, Date

Signature

(For Information: The statutory declaration is only necessary for a Master or Bachelor Thesis)